

1. INTEGRATED SITUATION

Project Description

LexiTech Solutions Ltd is a software development company based in Kigali City, specializing in building user-centric mobile applications that provide easy access to educational and reference resources.

LexiTech Solutions Ltd has recently been contracted to develop a **Dictionary Mobile Application** aimed at helping users quickly find word meanings, pronunciations, and usage examples. As part of the mobile application development team, your role is to design and implement a cross-platform mobile application that runs seamlessly on both Android and iOS platforms.

The Dictionary Mobile App will consume data from the Free Dictionary API available at:

```
https://api.dictionaryapi.dev/api/v2/entries/en/
```

The application should allow users to:

- Search for English words
- View definitions, parts of speech, and example sentences
- Listen to word pronunciations (where available)
- Access multiple meanings of a word
- Handle cases where a word is not found gracefully

The app must provide a clean, intuitive, and responsive user interface that follows modern mobile UI/UX standards. It should support efficient data retrieval from the API, display results in a structured and readable format, and ensure smooth navigation across different screens.

Basic functionalities such as search input handling, API data fetching, error handling, and result rendering must be implemented. The application should work reliably on both Android and iOS devices, offering users a fast and convenient way to explore word meanings anytime and anywhere.

Task

As a mobile application developer, you are tasked with developing the Dictionary Mobile App as described above, ensuring cross-platform compatibility, proper API integration, and a user-friendly experience.

Activities

Activity 1: Word Search & API Integration

- Design a search screen with a text input field and a search button
- Validate user input to ensure the search field is not empty
- Capture the entered word when the user submits the search field
- Construct the API request URL dynamically using the entered word
- Send an HTTP GET request to:

```
https://api.dictionaryapi.dev/api/v2/entries/en/{word}
```

- Display a loading indicator while the API request is in progress
- Receive and parse the JSON response from the API
- Store the fetched word data temporarily for display and navigation

Activity 2: Display Word Details

- Extract the main word, phonetics, meanings, and definitions from the API response
- Display the searched word prominently at the top of the screen
- Show phonetic spelling of the word
- Display each part of speech (e.g., noun, verb, adjective)
- List all definitions under their respective parts of speech
- Display example sentences when provided by the API
- Ensure the layout supports multiple meanings and long definitions
- Apply consistent styling and spacing for readability

Activity 3: Audio Pronunciation Feature

1. Check if an audio pronunciation URL exists in the API response
2. Display a pronunciation (speaker) icon next to the word or phonetics
3. Load the audio file from the provided URL
4. Play the audio when the user taps the pronunciation icon
5. Handle cases where multiple audio pronunciations are available.
6. Disable or hide the audio feature if no pronunciation is provided.
7. Manage audio playback states (play, pause, stop).

Activity 4: Drawer Navigation & Search History

- Implement a drawer navigation in the application
- Create a search history data structure to store previously searched words
- Add each successfully searched word to the history list
- Display the list of searched items in the drawer menu
- Allow users to tap a word from the drawer
- Trigger a new API request when a history item is selected
- Refresh the word detail screen with the selected word's data
- Prevent duplicate entries in search history

Activity 5: Error Handling & User Feedback

- Detect when the API returns a "word not found" response (e.g., 404)
- Display a clear and user-friendly "Word not found" message
- Handle network connectivity issues gracefully
- Show an error message when the API request fails
- Hide loading indicators when an error occurs
- Prevent the app from crashing due to malformed responses
- Allow users to retry the search after an error
- Display empty-state messages when no data is available

Instructions

- In the first hour, you should read carefully about the problem to be solved and write down appropriate designs including but not limited to:
 - Data Flow Diagram (DFD)
 - Application Architecture
 - API Endpoints
 - Required Pages/Screens
- Android applications should be built using **React Native**.
- Input must be validated where applicable.
- Handle errors and validations appropriately; users should receive meaningful messages whenever an error or exception occurs.
- Interaction with APIs should be done using the **Axios** library.
- Testing mobile applications should be done using the **Expo CLI** package.

Additional Information

Item	Details
Time Allowed	5 Hours
Equipment Provided	Computer and Telephone